```
#%% md
# 20 aprile



#%

#%
l1 = [1, 2, 3]
l2 = [1, 2, 3]
l1_alias = l1

print(hex(id(l1)))
print(hex(id(l1_alias)))
print(hex(id(l2)))
#%

#%
l1[0] =100
l1[2] =200

print(l1)
print(l1_alias)
#%
print(hex(id(l1))) #0x1061bd8c0
print(hex(id(l1_alias))) #0x1061bd8c0
#%
#



l1 = [1, 2, 3]
l1_alias = l1 #alias
print('prima della assegnazione')
print(hex(id(l1))) #
print(hex(id(l1_alias)))

l1_alias = [] # altra lista
print('dopo la assegnazione')
print(hex(id(l1))) #
print(hex(id(l1_alias)))
#%

```

```
l1 = [1, 2, 3]
l1_alias = l1 #alias
print('prima della assegnazione')
print(hex(id(l1))) #
print(hex(id(l1_alias)))

l1_alias = 10 # altro oggetto in memoria
print('dopo la assegnazione')
print(hex(id(l1))) #
print(hex(id(l1_alias)))
#%%

#%%
a = [1, 2, 3]
a_alias = a
b = [1, 2, 3]
# contenuto
print(a == a_alias)
print(a == b)
# identità
print(a is a_alias)
print(a is b)
#%%

#%%

#%%
a = list(range(10))
b = a[2:5]
c = a[2:5]
print(a)
print(b)
print(c)
print(b is c)

a [2] = 100
b [0] = 200
print(a)
print(b)
print(c)
#%%
a = [1,2, ['x','y','z'] ]
b = a [1:3] #[2, ['x', 'y', 'z']]
print(b)
```

```
print(a is b)
a [1] =200
print('dopo il cambiamento 1')
print(a)
print(b)
# ---
a[2][0] = 'W'
print('dopo il cambiamento 2')
print(a)
print(b)
#%%
a = [1,2, ['x','y','z'] ]
b = a [1:3] #[2, ['x', 'y', 'z']]
# ---
a[2] = 'W'
print('dopo il cambiamento 3')
print(a)
print(b)

#%%
# 22.aprile 2026
# ------
#%%
l1 = [1, [1,2,3],2,3,4,5,6,7,8,9,10]
l2 = l1[0:3]
print(l2[1],type(l2[1]))
l2[1].append(999)
print(l1)


#%%

#%%
l_alias = l1 #alias - stesso indirizzo
l2_shallow = l1[:] # copia superficiale - DIVERSO indirizzo
l2_shallo2= l1.copy()


#%%
import copy
l2_shallo2 = copy.copy(l1)

l_new= copy.deepcopy(l1) #completamente separata
```

```python

#%%

#%%
l1=[10, [1,2,3]]
l2 = l1
l3 = l1[:]
l4 = copy.deepcopy(l1)

l1[0] = 99
print('l1',l1) #[99, [1, 2, 3]]
print('l2',l2) #[99, [1, 2, 3]]
print('l3',l3) #[10, [1, 2, 3]]
print('l4',l4) #[10, [1, 2, 3]]



#%%


l1=[10, [1,2,3]]
l2 = l1
l3 = l1[:]
l4 = copy.deepcopy(l1)

l1[1].append(7)
print(l1) #[10, [1, 2, 3, 7]]
print(l2) #[10, [1, 2, 3, 7]]
print(l3) #[10, [1, 2, 3, 7]]
print(l4) #[10, [1, 2, 3]]
#%%

#%%

#%% md
# Ricapitolando:
#%%
# Ricapitolando

Lista1 = [1, 2, 3]
Lista2 = Lista1[:]   # shallow copy

print(id(Lista1))
print(id(Lista2))
```

```python

print(Lista1 is Lista2)  # verifica identità

#id(Lista1) e id(Lista2) sono diversi → oggetti distinti
# Lista1 is Lista2 restituisce False → non sono lo stesso oggetto

# Quindi la shallow copy crea una nuova lista,
# anche se inizialmente contiene gli stessi elementi.
#%%
# la shallow copy crea due liste distinte,
# ma condivide i riferimenti agli stessi oggetti interni.
# Quindi le liste sono indipendenti solo al primo livello

Lista1 = [[1, 2], [3, 4]]
Lista2 = Lista1[:]   # shallow copy

# Verifica identità delle liste
print(Lista1 is Lista2)          # False

# Verifica identità degli elementi interni
print(Lista1[0] is Lista2[0])    # True

# Modifica interna
Lista2[0][0] = 99

print(Lista1)  # cambia anche Lista1
print(Lista2)
# Le liste esterne sono oggetti diversi
# (is → False)
# Gli elementi interni sono gli stessi oggetti
# # (is → True)
# Modificare un oggetto mutabile interno (come una lista) si riflette in entrambe
#%% md
Una *shallow copy* in Python si comporta sempre allo stesso modo: crea una nuova lista ma copia solo i riferimenti agli oggetti contenuti, senza duplicarli, indipendentemente dal fatto che siano mutabili o immutabili. La differenza osservabile emerge perché gli oggetti mutabili possono essere modificati *in place*: in questo caso, la modifica è visibile attraverso entrambe le liste che condividono lo stesso riferimento. Gli oggetti immutabili (come tuple o stringhe), pur essendo anch'essi
```

condivisi, non possono essere modificati; ogni cambiamento comporta la creazione di un nuovo oggetto, quindi non produce effetti sull'altra lista.

```
#%% md
aa
aa



a
#%%

#%%
# 23 aprile

#%%
a= (1, [1,2,3], 10)
print(a[0])
a[1].append(10)
print(a)
#%%

#%%
a= (1, [1,2,3], 10)
print(hex(id(a[1])))
a[1][0] = 99
a[1].extend([10,20,30])
print(a)
print(hex(id(a[1])))
#%%

#%%
# add element to a tuple

t = (1,2,3)
# t.append(4)
tmp = list(t)
print(tmp, type(tmp))
tmp.append(4)
t = tuple(tmp)


#%%
```

```python
# DIZIONARI: key:value

d ={ 'nome':'ugo', 'cognome': 'fantini',  'eta': 19, (70, 19111): 'ing. navale'  }
print(d)
print(d[(70, 19111)])
#%%
d = dict(nome='ugo', eta =10 )
#%% md
'ugo' in d
#%%
d = dict(nome='ugo', eta =10) #, (1,1): 10 )
print('quale chiave vuoi cercare?')
k = input()
if k in d:
    print(d[k])
else:
    print('chiave non presente')


#%%


#%%
d = dict(nome='ugo', eta =10) #, (1,1): 10 )
print('quale chiave vuoi cercare?')
k = input()

v = d.get(k, None)
if v is None:
    print('chiave non presente')
#
#%%
v = d.get(k, None)
if v == 'non presente':
    print('chiave non presente')
#%%
# 29 aprile
#%%
d = dict() # d ={}

d['Cagliari'] = 30
d['Milano'] = 32
print(d)
```

```
#%%
print(d['Cagliari'])
#%%
temp = d['Cagliari']
print(temp)
#%%
temp = d['Caliari']
print(temp)
#%%
# gestire chiavi non presenti

k = 'Caliari'
if k in d: # è una chiave del d?
    t = d[k]
    print(t)
else:
    print('chiave non presente')


#%%
# gestire chiavi non presenti

k = 'Caliari'
d.get(k, None) # d[k] se k è presente

print(d.get(k, 'CIAO'))

#%%
print('dammi la chiave')
k = input() # stringa
t = d.get(k, None)
if t is None:
    print('chiave non presente')
else:
    print(t)


#%%
# VISTE


vista_valori=d.values()
vista_chiavi=d.keys()

```

```
print(vista_valori)
print(type(vista_valori))
print(vista_chiavi)

#%%

#%%
# accedo alle viste
# print(vista_valori[0]) non indicizzabile
# Posso trasformare VISTA in LISTA
lista_valori = list(vista_valori)
print(type(lista_valori))
print(lista_valori)
print(lista_valori[0])
lista_valori[0] =100


print(d)

#%%

#%%
# estrggo le viste e ricostruisco diz.
vista_valori = d.values()
vista_chiavi = d.keys()
# creare liste è superfluo ?
lista_valori = list(vista_valori)
lista_chiavi = list(vista_chiavi)

d1 ={}
i=0
while i < len(lista_valori):
    v = lista_valori[i]
    k = lista_chiavi[i]
    d1[k] = v
    i += 1
print(d1)




#%%
```

```python

#%%
# estrggo le viste e ricostruisco diz.- COL FOR
vista_valori = d.values()
vista_chiavi = d.keys()

d1 ={}

for k, v in zip (vista_chiavi, vista_valori):
    print(k, v)
    d1[k] = v

print(d1)

#%%
# for .. zip NON sono 2 for annidati!

# due FOR uno dentro l'altro sono questi:

l1 = [1,2,3]
l2 = [4,5,6]

for el1 in l1:
    for el2 in l2:
        print(el1, el2)

#%%
# FOR SU DIZIONARIO

#itera sulle chiavi
for k in d:
    v = d[k]
    print(k, v )
#%%
# FOR SU DIZIONARIO

#itera sulle chiavi
for k in d.keys():
    v = d[k]
    print(k, v )
#%%

#%%
# FOR SU DIZIONARIO
```

```python

#itera sui valori
for v in d.values():
    print(v)
    #print(d[v]) ERRATA


#%%
# dato diz temperature , trovare la lista di città
# con t < tmedia
#%%
# trasforma lettere in numeri

i = 2

if i == 1:
    print('uno')
elif i == 2:
    print('due')
elif i == 3:
    print('tre')


#%%
d={1:'uno', 2:'due', 3:'tre'}
i =20
print(d.get(i, 'valore non trovato'))


```